Admin Roles and Permissions

Overview

Yenkasa uses a rank-based role model with two overlapping concepts:

- staff roles
- public roles

This powers admin dashboards, moderation, analytics access, economy controls, an
d livestream creation eligibility.

Current implementation

Rank hierarchy

- unverified
- verified
- rising_star
- legend
- moderator
- admin
- junior_developer
- senior_developer

Key permission layers

- `middleware/permissions.js`: request-time rank and feature flags
- `models/permissions.model.js`: persisted permission records and helper statics
- `routes/roles.routes.js`: role dashboard, activation code generation, role ass
ignment/revocation
- `config/livestreamPermissions.js`: livestream eligibility rules

Role activation system

- generates one-time codes such as `YNK-STF-*` and `YNK-GEN-*`
- codes expire after 7 days
- public roles and staff roles are handled differently
- actor must outrank the role being granted

Known issues

- permission logic is duplicated between middleware, model statics, and config h
elpers
- role identity can come from `role`, `roleName`, `accessRole`, `staffRole`, or
`publicRoles`
- livestream permissions use a narrower rule set than the broader RBAC model

Scaling concerns

- none of the current permission checks are computationally expensive
- the real risk is policy drift, not raw performance

Recommended improvements

1. define one canonical role resolver
2. make all feature gating depend on the same authority source
3. document public-role versus staff-role precedence explicitly
4. add automated tests for privilege boundaries and role activation flows